

Wiring an Azure Functions MCP server into an Azure AI Foundry agent

A build record of connecting a C# Azure Functions app exposing two Model Context Protocol tools to a hosted Foundry agent running on `gpt-5-mini` — from project scaffold to a verified, traced tool call.

DATE	SUBSCRIPTION	REGION	AGENT	MODEL
29 July 2026	Azure subscription 1 (free trial)	East US	soubhik-demo-agent1	gpt-5-mini
PROTOCOL				
MCP over SSE				

01 · WHY THIS EXISTS

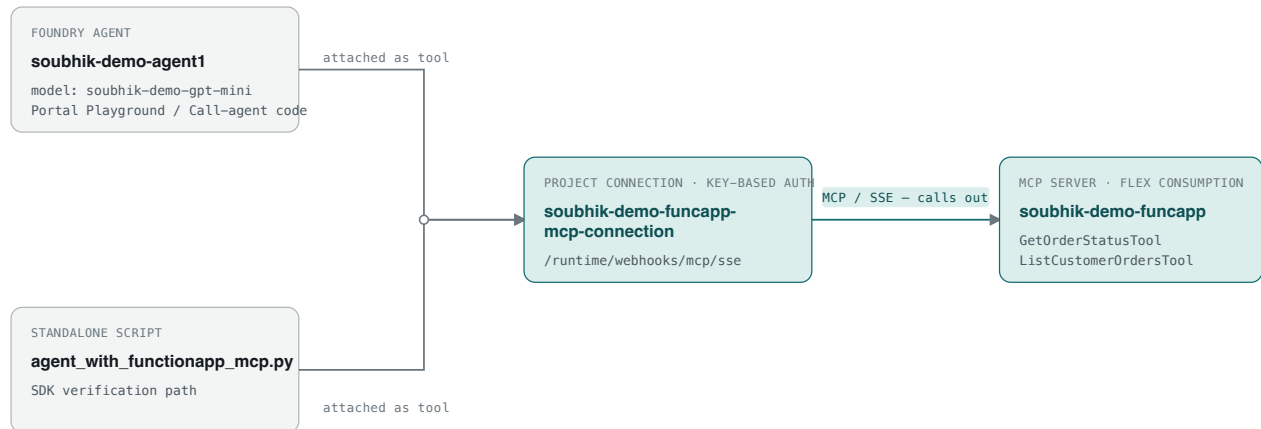
Objective

Give an Azure AI Foundry agent access to live order data by exposing that data as tools over the Model Context Protocol, rather than hard-coding the logic into the agent's prompt.

Concretely: stand up a C# Azure Functions app that speaks MCP, deploy it, register its endpoint as a tool connection inside a Foundry project, attach that connection to an agent built on a small, cost-efficient chat model, and prove the round trip works — the model should be able to decide, on its own, to call `GetOrderStatusTool` or `ListCustomerOrdersTool` mid-conversation and use the result to answer.

Architecture

The agent does not receive calls — it makes them. Both consumers on the left call out through the same project connection to the MCP server, which is not a separate service but the two functions already running inside the Function App on the right.



The agent (top left) and the standalone script (bottom left) each attach `soubhik-demo-funcapp-mcp-connection` as a tool. Neither talks to the Function App directly — the connection holds the endpoint and the function key once, and calls out over MCP/SSE to `soubhik-demo-funcapp`, whose two functions are the MCP server. Editable source: `architecture.drawio` (open in app.diagrams.net or the draw.io VS Code extension).

Resource reference

Every name used in this build, for reproducing or auditing the setup.

RESOURCE	NAME	NOTES
Resource group	soubhik-demo-rg	East US
Function App	soubhik-demo-funcapp	Linux, Flex Consumption, .NET 8 isolated worker
MCP tools	GetOrderStatusTool, ListCustomerOrdersTool	mcpToolTrigger, both Enabled
Foundry resource	soubhik-demo-foundry-1	Microsoft.CognitiveServices/acc kind AIServices
Foundry project	soubhik-demo-project	
Model deployment	soubhik-demo-gpt-mini	gpt-5-mini, 2025-08-07, GlobalStandard, capacity 10
Project connection	soubhik-demo-funcapp-mcp-connection	RemoteTool / CustomKeys, endpoints below
MCP endpoint	.../runtime/webhooks/mcp/sse	on soubhik-demo-funcapp.azurewebsites.net
Agent	soubhik-demo-agent1	kind: prompt · tools: web_search

Walkthrough

Nine steps, from scaffolding the MCP-capable Function project to inspecting the traced tool call the agent made against it.

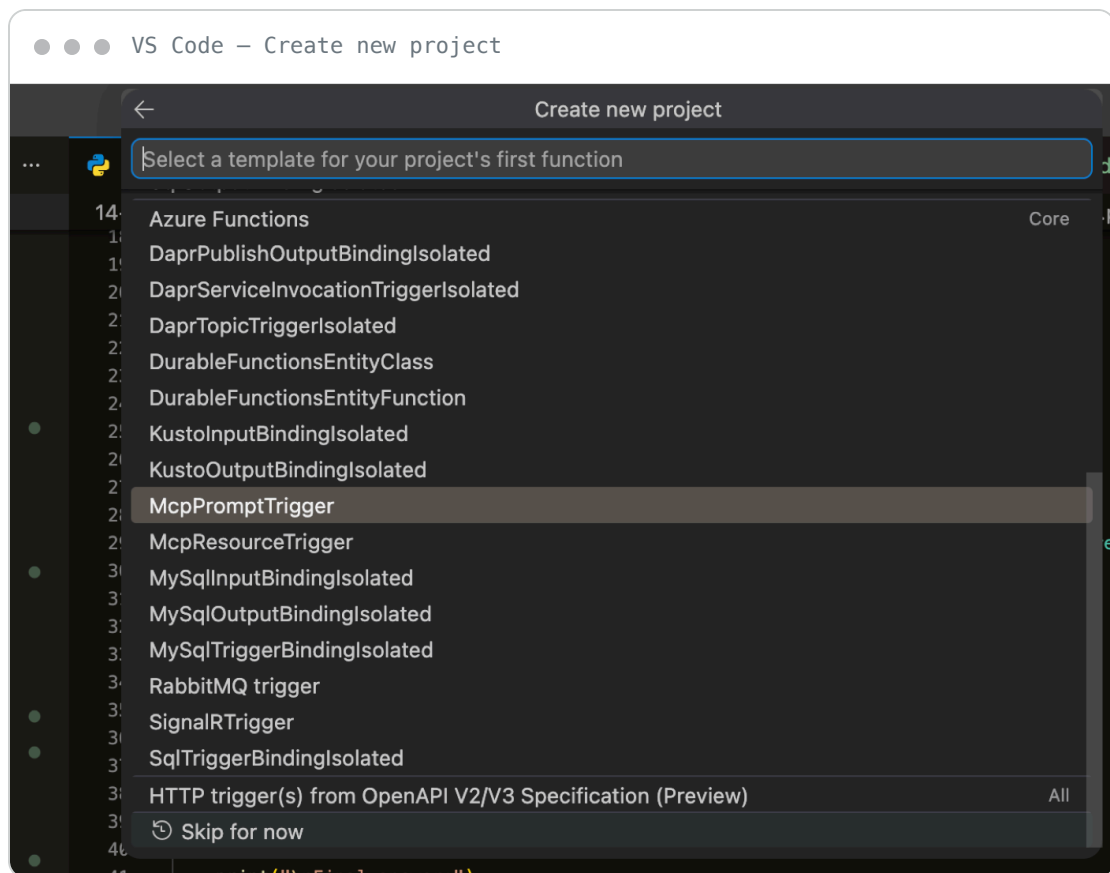
1

VS CODE · AZURE FUNCTIONS EXTENSION

Scaffold the MCP-capable Function project

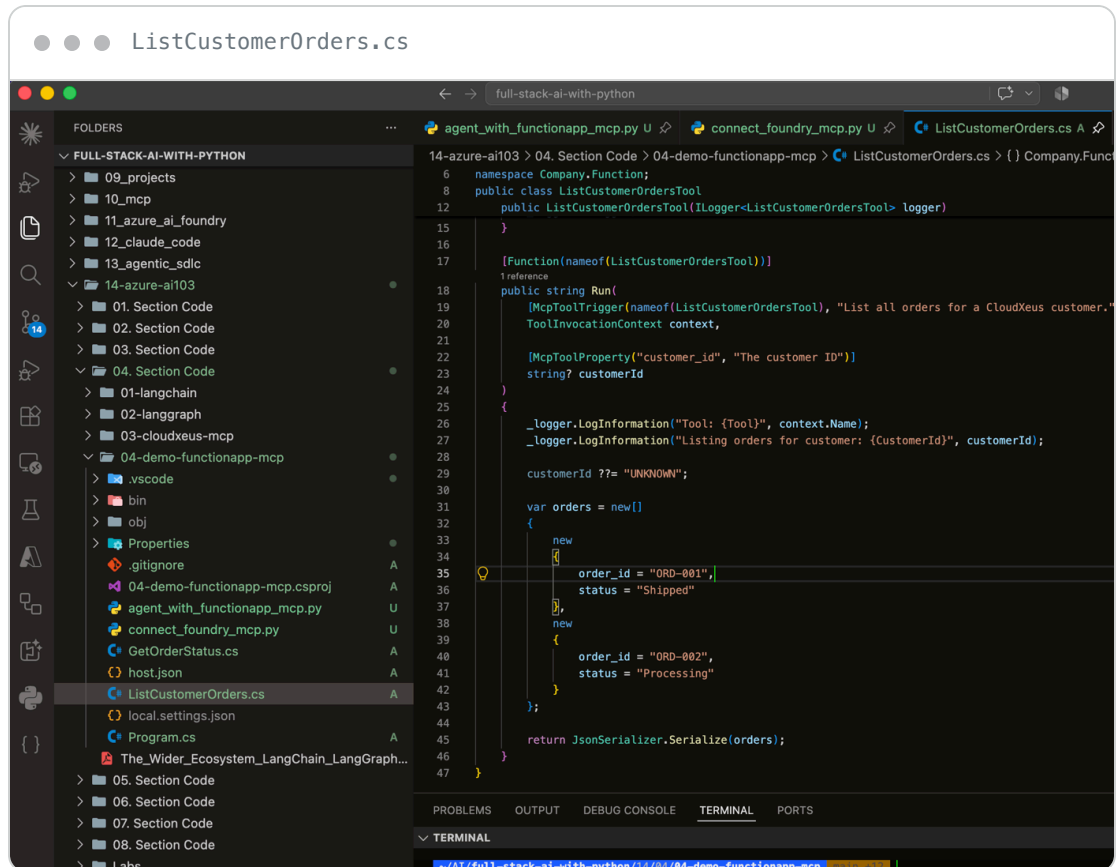
An Azure Functions project that will serve MCP tools starts the same way any Functions project does — **Create new project** — but the template list carries an MCP-specific family alongside the familiar HTTP and queue triggers:

McpToolTrigger (used by both functions in this project), **McpPromptTrigger**, and **McpResourceTrigger**. Picking one wires up the `Microsoft.Azure.Functions.Worker.Extensions.Mcp` package and the matching trigger attribute automatically.



Implement the second MCP tool

Each tool is an ordinary Function decorated with `[McpToolTrigger]` for its name and description, and `[McpToolProperty]` for each argument the model can pass in. `ListCustomerOrdersTool` takes a `customer_id` and returns a small hard-coded order list as JSON — enough to prove the round trip without a real database behind it.



```
14-azure-ai103 > 04. Section Code > 04-demo-functionapp-mcp > ListCustomerOrders.cs > {} Company.Func
6 namespace Company.Function;
8 public class ListCustomerOrdersTool
12 public ListCustomerOrdersTool(ILogger<ListCustomerOrdersTool> logger)
15 }
16
17 [Function(nameof(ListCustomerOrdersTool))]
18
19 public string Run(
20     [McpToolTrigger(nameof(ListCustomerOrdersTool), "List all orders for a CloudXeus customer."
21     ToolInvocationContext context,
22     [McpToolProperty("customer_id", "The customer ID")]
23     string? customerId
24 )
25 {
26     _logger.LogInformation("Tool: {Tool}", context.Name);
27     _logger.LogInformation("Listing orders for customer: {CustomerId}", customerId);
28
29     customerId ??= "UNKNOWN";
30
31     var orders = new[]
32     {
33         new
34         {
35             order_id = "ORD-001",
36             status = "Shipped"
37         },
38         new
39         {
40             order_id = "ORD-002",
41             status = "Processing"
42         }
43     };
44
45     return JsonSerializer.Serialize(orders);
46 }
47 }
```

Confirm the deployment is live

After `func azure functionapp publish`, the Portal is the fastest place to confirm the platform actually picked the code up: status **Running** on a **Flex Consumption** plan, and both `GetOrderStatusTool` and `ListCustomerOrdersTool` listed under **Functions** with trigger type `mcpToolTrigger` and status **Enabled**.

The screenshot shows the Azure Portal interface for a Function App named 'soubhik-demo-funcapp'. The left sidebar contains navigation options: Overview, Activity log, Access control (IAM), Tags, Diagnose and solve problems, Microsoft Defender for Cloud, Events (preview), AI (preview), Resource visualizer, Functions, App keys, Proxies, Deployment, Settings, Environment variables, and Configuration. The main area displays the 'Overview' tab for the Function App. It shows essential information: Resource group (soubhik-demo-rq), Status (Running), Location (East US), Subscription (Azure subscription 1), and Subscription ID (a2a03c7b-e15c-45c6-9ddd-dfa0552e2007). It also lists the Default domain (soubhik-demo-funcapp.azurewebsites.net), Operating System (Linux), Plan type (Flex Consumption), and Instance Memory (2048 MB). Below this, the 'Functions' tab is active, showing a table of functions. The table has columns for Name, Trigger, Status, and Monitor. Two functions are listed: 'GetOrderStatusTool' and 'ListCustomerOrdersTool', both with a trigger type of 'mcpToolTrigger' and a status of 'Enabled'.

Name	Trigger	Status	Monitor
GetOrderStatusTool	mcpToolTrigger	Enabled	Invocations and more
ListCustomerOrdersTool	mcpToolTrigger	Enabled	Invocations and more

Register the MCP endpoint as a project connection

Inside the Foundry project, **Tools** holds a connection pointing at the Function App's MCP endpoint, with **key-based** authentication so the function key never has to live in agent instructions or client code. The connection panel already shows it in use by one agent — `soubhik-demo-agent1`.

The screenshot displays the Microsoft Foundry interface for managing tools. The breadcrumb trail at the top reads 'Foundry - Tools - soubhik-demo-funcapp-mcp-connection'. The left sidebar contains a 'Create' section with links to Agents, Deployments, Services, Tools (highlighted), Knowledge, Guardrails, Memory, Data, and an 'Optimize' section with links to Evaluations and Fine-tune. The main content area shows the details for the 'soubhik-demo-funcapp-mcp-connection' tool, which was last updated on 7/29/2026. It includes buttons to 'Use in a toolbox', 'Use in an agent', and 'Delete tool'. Below the title, there are tabs for 'Details' and 'Docs'. The 'Details' tab shows 'No description available'. The 'Remote MCP server endpoint' is 'https://soubhik-demo-funcapp.azurewebsites.net/run', and the 'Authentication' is 'Key-based'. There is an 'Edit' button for the authentication. A checkbox for 'Governed with AI Gateway' is present and unchecked. The 'Project connection ID' is a long alphanumeric string. At the bottom, a table titled 'Used in agents' shows one agent, 'soubhik-demo-agent1', with a modified on date of '-'. The page number '1-1 of 1' and navigation links '< Prev Next >' are at the bottom.

Foundry - Tools - soubhik-demo-funcapp-mcp-connection

Microsoft Foundry / soubhik-demo-project

Search with AI (3K)

Create

- Agents
- Deployments
- Services
- Tools**
- Knowledge
- Guardrails
- Memory
- Data
- Optimize
 - Evaluations
 - Fine-tune

soubhik-demo-funcapp-mcp-connection Last updated: 7/29/2026

Use in a toolbox Use in an agent Delete tool

Details Docs

No description available

Remote MCP server endpoint Authentication

https://soubhik-demo-funcapp.azurewebsites.net/run Key-based Edit

☐ Governed with AI Gateway

Project connection ID

/subscriptions/a2a03c7b-e15c-45c6-9ddd-dfa0552e2007/resourceGroups/soubhik-demo-rg/providers/Microsoft.CognitiveServices/accounts/soubhik-demo-foundry-1/projects/soubhik-demo-project/connections/soubhik-demo-funcapp-mcp-connection

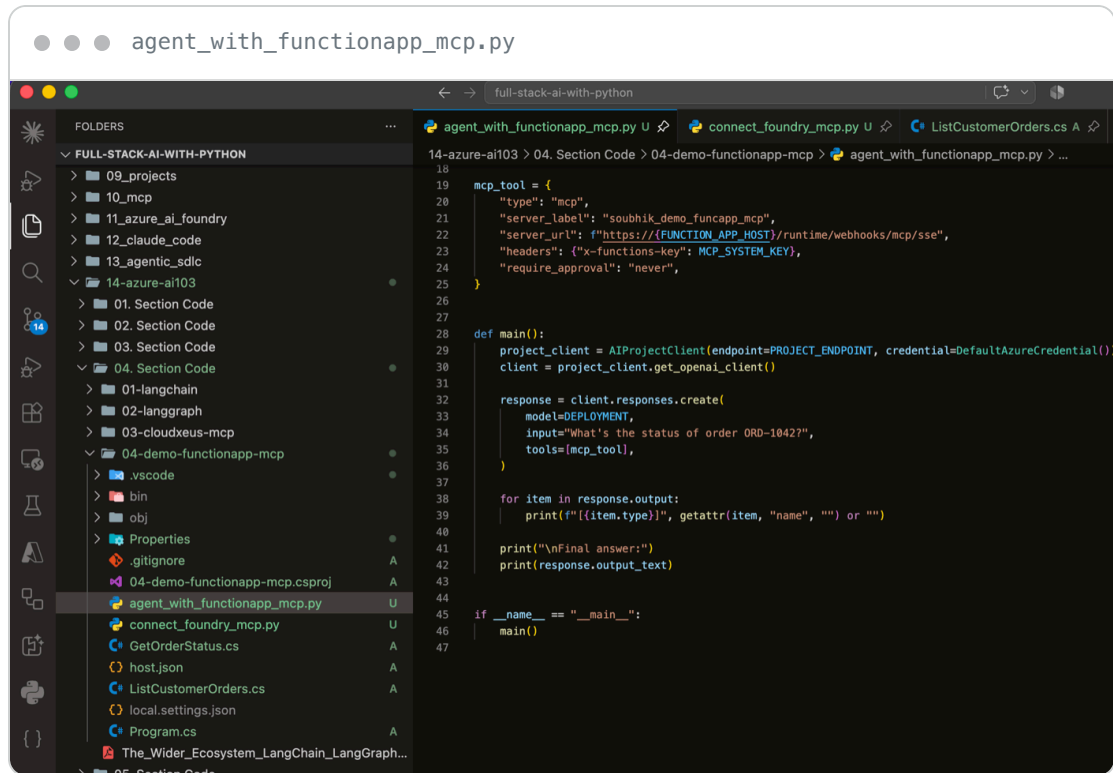
Used in agents

Name	Modified on
soubhik-demo-agent1	-

1-1 of 1 < Prev Next >

Verify the same tool from code

Alongside the Portal-defined agent, a standalone script confirms the connection works from the SDK side too — calling the deployment directly through the **Responses API** with the MCP tool attached inline, independent of any saved agent definition.



```
agent_with_functionapp_mcp.py

14-azure-ai103 > 04. Section Code > 04-demo-functionapp-mcp > agent_with_functionapp_mcp.py > ...
18
19 mcp_tool = {
20     "type": "mcp",
21     "server_label": "soubhik_demo_funcapp_mcp",
22     "server_url": f"https://{FUNCTION_APP_HOST}/runtime/webhooks/mcp/sse",
23     "headers": {"x-functions-key": MCP_SYSTEM_KEY},
24     "require_approval": "never",
25 }
26
27
28 def main():
29     project_client = AIProjectClient(endpoint=PROJECT_ENDPOINT, credential=DefaultAzureCredential())
30     client = project_client.get_openai_client()
31
32     response = client.responses.create(
33         model=DEPLOYMENT,
34         input="What's the status of order ORD-1042?",
35         tools=[mcp_tool],
36     )
37
38     for item in response.output:
39         print(f"[{item.type}]", getattr(item, "name", "") or "")
40
41     print("\nFinal answer:")
42     print(response.output_text)
43
44
45 if __name__ == "__main__":
46     main()
47
```

Define the agent

`soubhik-demo-agent1` is a `prompt`-kind agent on the `soubhik-demo-gpt-mini` deployment, with two tools attached: `web_search` and `mcp`. The `mcp` entry references the connection by label and reuses its `project_connection_id` — the server URL and auth live on the connection, not duplicated into the agent definition.

Foundry – soubhik-demo-agent1 – YAML

Chat **YAML** Call agent

9114t ⓘ ⋮ Metri

```
1 metadata:
2   logo: Avatar_Default.svg
3   microsoft.voice-live.enabled: "false"
4   object: agent.version
5   id: soubhik-demo-agent1:1
6   name: soubhik-demo-agent1
7   version: "1"
8   description: ""
9   created_at: 1785319470
10 definition:
11   kind: prompt
12   model: soubhik-demo-gpt-mini
13   instructions: ""
14   tools:
15     - type: web_search
16     - type: mcp
17       server_label: soubhik-demo-funcapp-mcp-connection
18       server_url: https://soubhik-demo-funcapp.azurewebsites.net/runtime/webhooks/mcp/sse
19       project_connection_id: soubhik-demo-funcapp-mcp-connection
20   draft: false
21   status: active
22   instance_identity:
23     principal_id: 16755f7f-2a56-4601-bb88-085543187ffd
24     client_id: 16755f7f-2a56-4601-bb88-085543187ffd
25   blueprint:
26     principal_id: c042bbce-4474-4341-a5c7-b09d48bcc0a2
27     client_id: bb64df13-9ee9-49f2-8907-747b383b7939
28   blueprint_reference:
29     type: ManagedAgentIdentityBlueprint
30     blueprint_id: soubhik-demo-agent1-1e8df
31   agent_guid: 1e8df6e4-28d4-4a20-a2e5-4c90dcc63b61
32
```

Get the caller code

Foundry generates a ready-to-run snippet for calling this exact agent: open an

`AIProjectClient`, get its OpenAI-compatible client, and call

`responses.create()` with an `agent_reference` in `extra_body` naming the agent and version. No tool wiring needed in the call itself — the agent definition already carries it.

● ● ● Foundry – soubhik-demo-agent1 – Call agent

Chat YAML **Call agent** 9114t Metrics ⚙️ +

Send a message to this agent and get a response.

Agent name soubhik-demo-agent1 Project endpoint https://soubhik-demo-foundry-1.servi

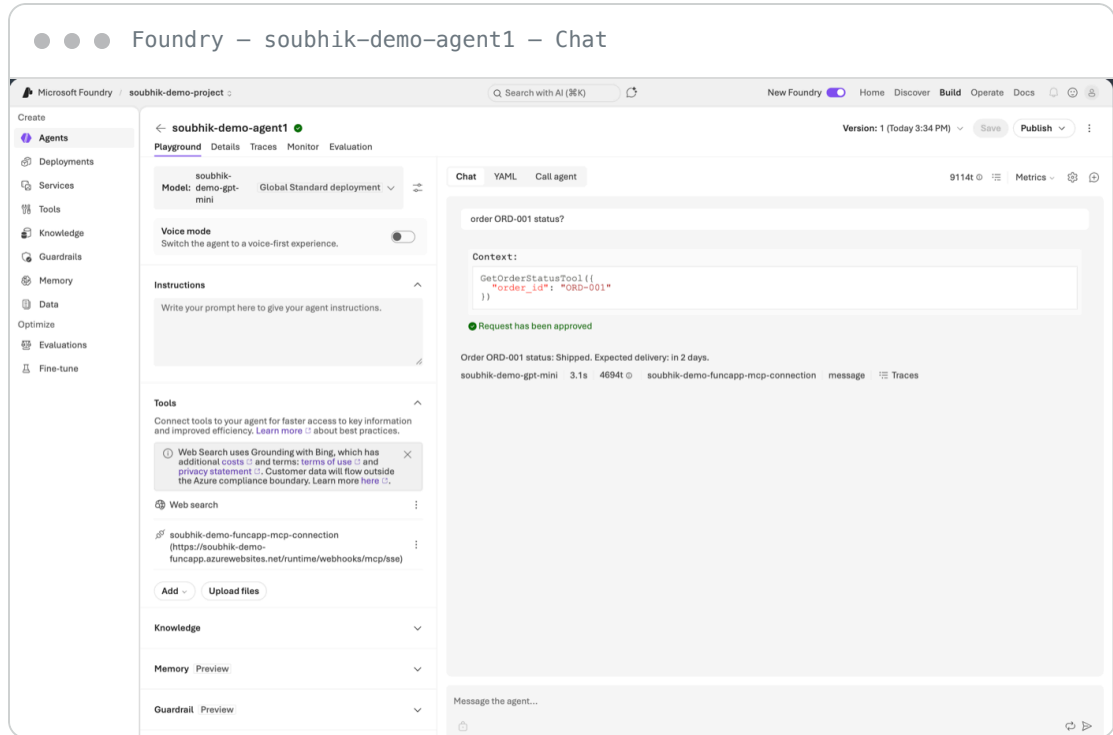
Call this agent

Language Python ▾

```
1 # Before running the sample:
2 # pip install azure-ai-projects>=2.1.0
3
4 from azure.identity import DefaultAzureCredential
5 from azure.ai.projects import AIProjectClient
6
7 endpoint = "https://soubhik-demo-foundry-1.services.ai.azure.com/api/projects/soubhik-demo-project"
8
9 project_client = AIProjectClient(
10     endpoint=endpoint,
11     credential=DefaultAzureCredential(),
12 )
13
14 my_agent = "soubhik-demo-agent1"
15 my_version = "1"
16
17 openai_client = project_client.get_openai_client()
18
19 # Reference the agent to get a response
20 response = openai_client.responses.create(
21     input=[{"role": "user", "content": "Tell me what you can help with."}],
22     extra_body={"agent_reference": {"name": my_agent, "version": my_version, "type": "agent_reference"}},
23 )
24
25 print(f"Response output: {response.output_text}")
```

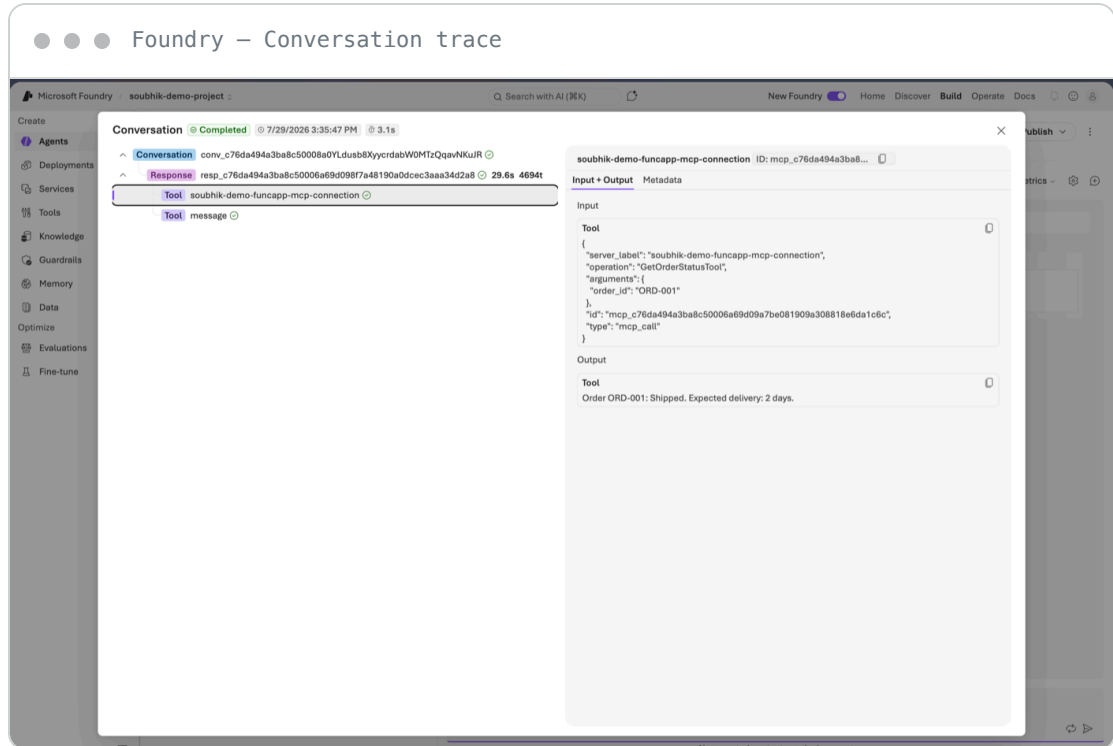
Test it live

A direct question — "order ORD-001 status?" — is enough to trigger the model to call `GetOrderStatusTool` with `{"order_id": "ORD-001"}`, get back the tool's hard-coded response, and fold it into a plain-language answer. The Playground surfaces the tool call as inline context before the final message.



Inspect the trace

The conversation trace breaks the turn into its constituent calls: a **Tool** step against `soubhik-demo-funcapp-mcp-connection` carrying the exact MCP call payload (`operation: GetOrderStatusTool`, `arguments: {"order_id": "ORD-001"}`) and its raw output, followed by the **message** step that turns it into prose. This is the same call the Function App's own logs would show on the other side.



Verification transcript

The exact turn from step 8 and 9, reconstructed from the trace.

USER

order ORD-001 status?

SOUBHIK-DEMO-AGENT1 · TOOL CALL

soubhik-demo-funcapp-mcp-connection.GetOrderStatusTool({ "order_id": "ORD-001" }) →
"Order ORD-001: Shipped. Expected delivery: 2 days."

SOUBHIK-DEMO-AGENT1

Order ORD-001 status: Shipped. Expected delivery: in 2 days.

model: soubhik-demo-gpt-mini duration: 3.1s tokens: 4694

tool: soubhik-demo-funcapp-mcp-connection

Platform findings

Four things that were not obvious going in, each of which changed how the connection had to be built.

01

Owner does not imply Foundry data-plane access RBAC

Being subscription **Owner** was not enough to create or run an agent on a fresh Foundry resource — every call failed with `PermissionDenied`. Classic control-plane roles (Owner, Contributor) do not carry Cognitive Services *data actions*; the account needed an explicit **Azure AI Developer** role assignment scoped to the Foundry resource before agent creation would succeed.

02

The smallest model is a moving target MODEL LIFECYCLE

`gpt-4o-mini` and `gpt-4.1-nano` are both past their new-deployment cutoff, and this subscription carried zero quota for any nano-tier model regardless of generation. `gpt-5-mini` was the smallest tier with real, non-zero quota — the actual floor is a function of subscription quota, not model catalog size.

03

MCP tools need the Responses API, not Assistants API SURFACE

The classic threads/runs Agent Service rejected an `mcp`-type tool outright: *"This model only supports Responses API compatible tools."* Tool-calling against an MCP server on this model family only works through `responses.create()` — which is also why the agent built in the Portal and the standalone script both end up going through the same OpenAI-compatible client rather than the Assistants-style SDK.

04

The mcp tool object is narrower than it looks SCHEMA

Neither `headers` nor `require_approval` is accepted inline on the tool definition at agent-creation time — both calls failed with `unknown_parameter`. Authentication has to be attached to the **connection** itself (the Portal's key-based auth field), or passed separately via `tool_resources` at run time when calling through the SDK directly.

Code appendix

Complete listings for every file referenced above.

GetOrderStatus.cs

C# · AZURE FUNCTIONS ISOLATED WORKER

```
using System.Text.Json;
using Microsoft.Azure.Functions.Worker;
using Microsoft.Azure.Functions.Worker.Extensions.Mcp;
using Microsoft.Extensions.Logging;

namespace Company.Function;

public class GetOrderStatusTool
{
    private readonly ILogger<GetOrderStatusTool> _logger;

    public GetOrderStatusTool(ILogger<GetOrderStatusTool> logger)
    {
        _logger = logger;
    }

    [Function(nameof(GetOrderStatusTool))]
    public string Run(
        [McpToolTrigger(nameof(GetOrderStatusTool), "Get the current status of a CloudXeus order.")]
        ToolInvocationContext context,

        [McpToolProperty("order_id", "The order ID")]
        string? orderId
    )
    {
        _logger.LogInformation("Tool: {Tool}", context.Name);
        _logger.LogInformation("Order lookup: {OrderId}", orderId);

        orderId ??= "UNKNOWN";

        return $"Order {orderId}: Shipped. Expected delivery: 2 days.";
    }
}
```

```
using System.Text.Json;
using Microsoft.Azure.Functions.Worker;
using Microsoft.Azure.Functions.Worker.Extensions.Mcp;
using Microsoft.Extensions.Logging;

namespace Company.Function;

public class ListCustomerOrdersTool
{
    private readonly ILogger<ListCustomerOrdersTool> _logger;

    public ListCustomerOrdersTool(ILogger<ListCustomerOrdersTool> logger)
    {
        _logger = logger;
    }

    [Function(nameof(ListCustomerOrdersTool))]
    public string Run(
        [McpToolTrigger(nameof(ListCustomerOrdersTool), "List all orders for a CloudXeus customer.")]
        ToolInvocationContext context,

        [McpToolProperty("customer_id", "The customer ID")]
        string? customerId
    )
    {
        _logger.LogInformation("Tool: {Tool}", context.Name);
        _logger.LogInformation("Listing orders for customer: {CustomerId}", customerId);

        customerId ??= "UNKNOWN";

        var orders = new[]
        {
            new
            {
                order_id = "ORD-001",
                status = "Shipped"
            },
            new
            {
                order_id = "ORD-002",
                status = "Processing"
            }
        };

        return JsonSerializer.Serialize(orders);
    }
}
```

```
import requests
from azure.identity import DefaultAzureCredential, get_bearer_token_provider

credential = DefaultAzureCredential()

PROJECT_RESOURCE_ID = (
    "/subscriptions/<subscription-id>"
    "/resourceGroups/soubhik-demo-rg"
    "/providers/Microsoft.CognitiveServices/accounts/soubhik-demo-foundry-1"
    "/projects/soubhik-demo-project"
)

PROJECT_CONNECTION_NAME = "soubhik-demo-funcapp-mcp-connection"

FUNCTION_APP_HOST = "soubhik-demo-funcapp.azurewebsites.net"
MCP_SYSTEM_KEY = "<function-app-system-key>" # from: az functionapp keys list

mcp_endpoint = f"https://{FUNCTION_APP_HOST}/runtime/webhooks/mcp/sse"

bearer_token_provider = get_bearer_token_provider(
    credential, "https://management.azure.com/.default"
)

headers = {
    "Authorization": f"Bearer {bearer_token_provider()}",
    "Content-Type": "application/json"
}

response = requests.put(
    f"https://management.azure.com{PROJECT_RESOURCE_ID}/connections/{PROJECT_CONNECTION_NAME}"
    "?api-version=2025-10-01-preview",
    headers=headers,
    json={
        "name": PROJECT_CONNECTION_NAME,
        "type": "Microsoft.MachineLearningServices/workspaces/connections",
        "properties": {
            "authType": "CustomKeys",
            "category": "RemoteTool",
            "target": mcp_endpoint,
            "isSharedToAll": True,
            "credentials": { "keys": { "x-functions-key": MCP_SYSTEM_KEY } },
            "metadata": { "ApiType": "Azure", "McpTransport": "sse" }
        }
    }
)

response.raise_for_status()
print(f"Connection '{PROJECT_CONNECTION_NAME}' created or updated successfully.")
```

```
from azure.ai.projects import AIProjectClient
from azure.identity import DefaultAzureCredential

PROJECT_ENDPOINT = "https://soubhik-demo-foundry-1.services.ai.azure.com/api/projects/soubhik-demo-project"
DEPLOYMENT = "soubhik-demo-gpt-mini"

FUNCTION_APP_HOST = "soubhik-demo-funcapp.azurewebsites.net"
MCP_SYSTEM_KEY = "<function-app-system-key>"

mcp_tool = {
    "type": "mcp",
    "server_label": "soubhik_demo_funcapp_mcp",
    "server_url": f"https://{FUNCTION_APP_HOST}/runtime/webhooks/mcp/sse",
    "headers": { "x-functions-key": MCP_SYSTEM_KEY },
    "require_approval": "never",
}

def main():
    project_client = AIProjectClient(endpoint=PROJECT_ENDPOINT, credential=DefaultAzureCredential())
    client = project_client.get_openai_client()

    response = client.responses.create(
        model=DEPLOYMENT,
        input="What's the status of order ORD-1042?",
        tools=[mcp_tool],
    )

    for item in response.output:
        print(f"[{item.type}]", getattr(item, "name", "") or "")

    print("\nFinal answer:")
    print(response.output_text)

if __name__ == "__main__":
    main()
```

Conclusion

The integration works end to end and is reproducible

A Foundry agent running on a quota-available small model can now discover and call two live MCP tools hosted on a deployed Azure Function App, using a project-level connection as the single point of trust for authentication. The trace viewer confirms the exact call, arguments, and raw tool output for every turn — this is verifiable, not assumed.

2

MCP tools live on soubhik-demo-funcapp

1

Project connection mediating auth

3.1s

Round trip on the verified test turn

What is out of scope here, and worth flagging before this goes further than a demo: the tool data is hard-coded (no real order database behind it), `require_approval` is set to `never` for convenience rather than a considered production policy, and the agent has no conversation memory across turns — each Responses API call is independent unless `previous_response_id` chaining is added deliberately.